

Reasoning?

DD2459 Software Reliability 2022

Take-home Exam

**Tuesday 15 March, 15.00 – Thursday 17 March, 12.00 midday
(Version 1)**

Instructions.

Please read the following instructions carefully.

- (1) Clearly mark at the top of each sheet you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) Your work should be uploaded in one .pdf file to the Course Assignments page on Canvas no later than Thursday 17 March 2022 at 12.00 midday. After this time, it will be marked as late and points will be subtracted.
- (4) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm! You can also e-mail the examiner at karlm@kth.se.
- (5) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates to this document that will have new version numbers.
- (6) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (7) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

Please turn over.

NOTE: For a full score of 85 points you should answer all 4 questions below.

Question 1. (Total 15 points)

Write appropriate pre- and post-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid under-specification, i.e. write all the constraints necessary for each condition and do not omit any relevant constraints.

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.

You do not have to provide Java code for any of the operations, only JML.

- (i) **(5 points)** A root mean square function (aka. RMS):

```
float rms(float[] sample)
```

Input: A one-dimensional array `sample` of `float` (the data set).

Output: The `float` result which should be the square root of the arithmetic mean of the squares of all `sample` values.

Note: Your JML specification should consider the possibility that floating point operations are not always perfectly accurate.

Model Answer:

This is a typical mathematical specification that must take floating point accuracy into account in some way, e.g.

```
requires sample != null
```

```
ensures
```

```
Java.Lang.Math.abs(\result -  
Java.Lang.Math.sqrt( (\sum int i; 0 <= i <  
\old(sample).length; \old(sample[i]) * \old(sample[i]))  
/ \old(sample).length  
) < epsilon
```

- (ii) **(5 points)** A reverse concatenation function:

```
char[] rconcat(char[] string1, char[] string2)
```

Input: Two arrays `string1` and `string2` of characters.

Output: The result of concatenating the reverse of `string2` followed by the reverse of `string1`, i.e. `reverse(string2).reverse(string1)`.

Model Answer:

```
requires string1 != null & string2 != null

ensures
  \result != null &&
  \result.length == \old(string1).length +
  \old(string2).length &&

  (\forall int i; 0 <= i < \old(string2).length;
  \result[i] == \old(string2)[string2.length - (i+1)])
  &
  (\forall int i; 0 <= i < \old(string1).length;
  \result[i + \old(string2).length] ==
  \old(string1)[ \old(string1).length - (i+1)])
```

(iii) (5 points) A random shuffle function:

```
char[] rconcat(char[] string1)
```

Input: An array `string1` of characters.

Output: A string of characters that corresponds to some random shuffle of the characters of `string1`.

Model Answer: This is similar to a specification of sorting, but without the requirement for a linear ascending or descending sequence.

```
requires string1 != null

ensures
  \forall int j; 0 <= j < \old(string1).length;
  (\num_of int i; 0 <= i < \result.length;
  \result[i] == \old(string1)[j])
  ==
  (\num_of int i; 0 <= i < \old(string1).length;
  \old(string1)[i] == \old(string1)[j]) (2)
```

i.e. for each position j the number of $A[i]$'s equal to $\text{old}(A[j])$ after computation is the same as the number of $A[i]$'s equal to $\text{old}(A[j])$ before computation.

```
&&
\result.length == \old(string1).length (3)
```

Question 2. (Total 6 points)

- (i) (3 points) Write down one metamorphic equation that should be satisfied by any sorting function:

```
int[] sort(int[] myIntList)
```

Input: A one-dimensional array `myIntList` of `int` (the data set).

Output: A one-dimensional array of integers in ascending order.

Clearly define the data transformation T used on the input of your equation.

Model Answer: The sorting function satisfies any permutative law (similar to the inclusive law mentioned in Lecture 6) such as

$\text{sort}([x_1, \dots, x_n]) = \text{sort}(T([x_1, \dots, x_n]))$

where T is any permutation, for example:

$T([x_1, \dots, x_n]) = [x_n, \dots, x_1]$ is the reversal permutation

$T([x_1, \dots, x_n]) = [x_n, x_1, \dots, x_{n-1}]$ is the shift-1 right permutation

- (ii) (3 points) Starting from any concrete seed test case (i.e. input array values), write down as many distinct test cases as you can generate using the data transformation T that you defined in your answer to part (i). Is there any finite limit on the number of different test cases you can generate this way? If so, explain what is the limiting factor.

Model Answer: Given a seed test case such as $[1, \dots, n]$ a new test case with reversal would be $T[1, \dots, n] = [n, \dots, 1]$.

However, if we try to apply the reversal permutation T twice then we get $T^2[1, \dots, n] = [1, \dots, n]$. So higher powers of reversal $T^n[1, \dots, n]$ are just one of these two vectors and we only get two distinct test cases.

Notice that shift-1 right gives n new test cases and in general other permutations give different numbers of test cases, but no fixed permutation gives more than n . Note also that some permutations can give no new tests, depending also on the seed test, e.g. reversal of a symmetric vector!

Question 3. (Total 30 points) Recall that a *stack* is a linear data structure with a *last-in-first-out* (LIFO) access strategy. A stack has two operations `push(x)` which places an element x on the top of the stack, and `pop()` which removes the top element of the stack.

A *Boolean stack* is a stack that stores only the Boolean values $0, 1 : \text{Boolean}$. Like the Boolean shift register, we can model the Boolean stack as a *finite state machine* (FSM) if we assume that the input alphabet has three symbols: $\Sigma = \{\text{push}(0), \text{push}(1), \text{pop}()\}$.

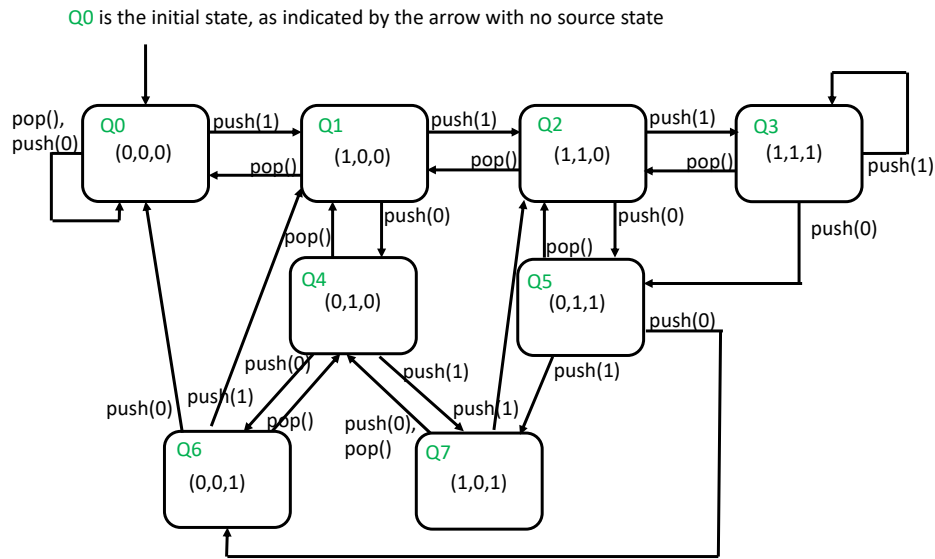
A *3-bit Boolean stack* has finite size, and stores exactly 3 Boolean values. For simplicity, we can handle *stack overflow* by allowing the value of the bottom element of the stack to be silently lost. We can assume that the *initial output value* of the 3-bit Boolean stack is $(0, 0, 0)$.

We can write out LTL formulas for a 3-bit Boolean stack, as formulas over the Boolean variables: `bit1`, `bit2`, `bit3`, `input`, where `bit1` is the top of the stack and `bit3` is the bottom. For example:

$$G(\text{input}=\text{pop}() \Rightarrow X(\text{bit3}=0)) \quad (\text{LTL Formula 1})$$

- (a) **(12 points)** Draw a *finite state Moore machine* (FSM) (i.e. outputs are associated with states) to model a 3-bit Boolean stack, as specified above. **Important Note:** Your FSM solution must satisfy LTL Formula 1 above. What does LTL Formula 1 mean, and why is it necessary? Carefully define your *state set* $Q = \{q_0, \dots, q_n\}$ by labelling all states with names, and clearly indicate the *initial state*. Carefully define the *state transition function* $\delta : Q \times \Sigma \rightarrow Q$ by means of labelled transitions between states. Carefully define the *output function* $\lambda : Q \rightarrow \{0,1\} \times \{0,1\} \times \{0,1\}$ by means of labelled states. You can assume 1 encodes true and 0 encodes false. What are the minimum and maximum number of states needed? Carefully explain your answer.

Model Answer: The formula means that whenever we pop the stack then in the next state `bit3` which cannot receive a shifted value from the right, as there is no `bit4`, instead gets value 0 by default. This ensures the stack returns to its initial state when all elements are popped off the stack.



The maximum number of states needed is clearly $2^3 = 8$, as the solution above shows. How do we know that there is no smaller FSM solution than the one above, i.e. with fewer states? The minimum number of states is also 8. The reason for this is that with any fewer states than 8 the FSM would confuse at least two distinct 3 bit input sequences, i.e. it would give the same output $\lambda(q_i) = \lambda(q_j)$ for different states q_i, q_j arising from different input sequences.

- (b) (9 points) For your FSM solution to part (a), write out trap conditions as LTL formulas such that the corresponding set of counterexamples would give 100% node coverage (NC) of your FSM. For each trap condition, write out a test case as a counterexample for it. You should try to find a minimal (i.e. redundancy free) test suite NC-TS that gives 100% node coverage. For each test case, you should predict the expected output according to your FSM model. Write up the test cases and the predicted outputs from your model in table format (as in Lab 3).

Model Answer: We will assume 3 Boolean variables x_1, x_2, x_3 and 1 state variable $state$ (with values $Q_0, \dots, Q_6$) which act to store the stack and state values. Below the bit vector is (bit1,bit2,bit3)

Now Q_3 can only be reached through Q_2 so

NC-TR1: $G(\neg state = Q_3)$

If Q_2 is accessed then Q_1 must have been accessed, as there is only 1 way of out Q_0 . So any test case satisfying NC-TR1 must cover states Q_0, Q_1, Q_2 and Q_3 .

The states Q_4, Q_5, Q_6 and Q_7 do not have unique access paths, so each needs its own trap formula

NC-TR2: G(!State=Q4)
 NC-TR3: G(!State=Q5)
 NC-TR4: G(!State=Q6)
 NC-TR5: G(!State=Q7)

NC-TR1	T=0	T=1	T=2	T=3	T=4
Input	push(1)	push(1)	push(1)		
Output	(0,0,0)	(1,0,0)	(1,1,0)	(1,1,1)	

NC-TR2	T=0	T=1	T=2	T=3	T=4
Input	push(1)	push(0)			
Output	(0,0,0)	(1,0,0)	(0,1,0)		

NC-TR3	T=0	T=1	T=2	T=3	T=4
Input	push(1)	push(1)	push(0)		
Output	(0,0,0)	(1,0,0)	(1,1,0)	(0,1,1)	

NC-TR4	T=0	T=1	T=2	T=3	T=4
Input	push(1)	push(0)	push(0)		
Output	(0,0,0)	(1,0,0)	(0,1,0)	(0,0,1)	

NC-TR5	T=0	T=1	T=2	T=3	T=4
Input	push(1)	push(0)	push(1)		
Output	(0,0,0)	(1,0,0)	(0,1,0)	(1,0,1)	

There are however several single test cases that give 100%NC,
 e.g:

tc1 = push(1) , push(1) , push(1) , push(0) ,push(1) ,push(0) ,push(0)

tc2 = push(1) , push(1) , push(1) , push(0) ,push(0) ,pop() ,push(1)

tc3 = push(1) , push(1) , push(1) , push(0) ,push(1) ,pop() ,push(0)

which visit every state. You can put one of these in a table for yourself.

- (c) **(6 points)** Show that edge coverage EC for your FSM of part (a) is stronger than node coverage **NC** by writing down three trap properties as LTL formulas which are not satisfied by your test suite *NC-TS* for part (b). For each of these three EC trap properties write down a test case as a counterexample for it. Present your three test cases and the predicted outputs from your model in table format as in part (b) above.

Model Answer: Since there are alternative paths to states Q4, Q5, Q6 and Q7, we can use edge coverage to enforce that the test case takes a specific path.

EC-TR1: $G(\neg \text{state} = \text{Q7} \ \& \ \text{input} = \text{pop}() \rightarrow X(\neg \text{state} = \text{Q4}))$

EC-TR2: $G(\neg \text{state} = \text{Q5} \ \& \ \text{input} = \text{push}(0) \rightarrow X(\neg \text{state} = \text{Q6}))$

EC-TR3: $G(\neg \text{state} = \text{Q5} \ \& \ \text{input} = \text{push}(1) \rightarrow X(\neg \text{state} = \text{Q7}))$

- (d) **(3 points)** Write down one LTL formula which exactly captures the intended behavior of the 3-bit Boolean stack. You should use Boolean variables to shorten your formula length.

Model Answer: Similarly to the Boolean queue we can write down a behavioral model as an LTL formula:

```
G(input = pop() => X(bit3 = 0) & (X(bit1) <-> bit2)
& (X(bit2) <-> bit3) ) &
G(input = push(1) => X(bit1 = 1) & (X(bit2) <-> bit1)
& (X(bit3) <-> bit2) ) &
G(input = push(0) => X(bit1 = 0) & (X(bit2) <-> bit1)
& (X(bit3) <-> bit2) )
```

Question 4. (Total 34 points)

- (1) Danny Daydream dreamt of implementing the following integer polynomial in two integer variables $f: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ where

$$f(x,y) = 3xy$$

and x, y are any integer values.

Here is Danny's code (note : $x, y: \text{int}$)

```
z = y;
x = 3*y*x;
w = x;
while (!(x == 0)) do {
    x = x - 2;
```



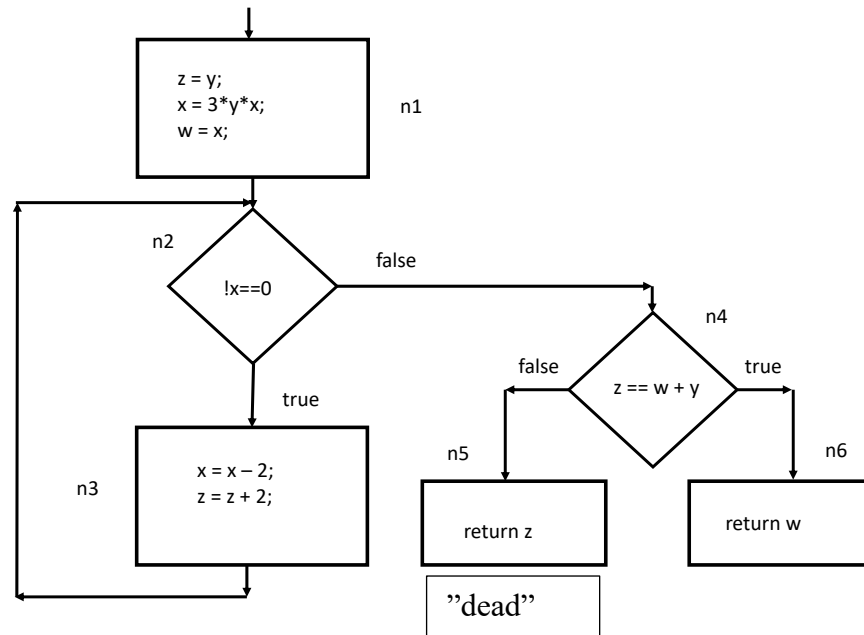
```

        z = z + 2;
    }
    if (z == w + y) then return w else return z;

```

(a) **(2 points)** Draw a condensation graph for Danny's code.

Model Answer:



(b) **(2 points)** Danny reasoned that just two glass-box test requirements would give 100% node coverage (NC) by subsuming any others. Define two such test requirements using your own condensation graph for part (a).

Model Answer: Two test requirements could be

NC-TR1: n1, n2, n4, n6

NC-TR2: n1, n2, n3, n2, n4, n5

which (if satisfiable) would obviously give 100%NC.

(c) **(15 points)** Using the method of symbolic program execution, for each of your two NC test requirements in part (b) derive a test case which satisfies it, or else show that no test case exists.

Model Answer: First we consider NC-TR1: n1, n2, n4, n6

Node	Instruction	Constraint
N1.a	$z = y;$	$z' = y$
N1.b	$x = 3*y*x;$	$x' = 3*y*x$
N1.c	$w=x;$	$w' = 3*y*x$
N2	$!x==0$	$x' = 0 \Leftrightarrow$

		$3*y*x = 0$
N4	$z == w+y$	$z' = w' + y' \Leftrightarrow$
		$y = 3*y*x + y \ \& \ 3*y*x = 0 \Leftrightarrow$
		$3*y*x = 0$
N6	return w	return $3*y*x$

Test cases have the form: $(x=0, y=\text{any integer})$ or $(y=0, x=\text{any integer})$

Next we consider NC-TR2: n1, n2, n3, n2, n4, n5

Node	Instruction	Constraint
N1.a	$z = y;$	$z' = y$
N1.b	$x = 3*y*x;$	$x' = 3*y*x$
N1.c	$w=x;$	$w' = 3*y*x$
N2	$!x==0$	$x' != 0 \Leftrightarrow$
		$3*y*x != 0$
N3.a	$x = x-2;$	$x' = 3*y*x - 2$
N3.b	$z = z+2$	$z' = y+2$
N2	$!x==0$	$3*y*x != 0 \ \& \ 3*y*x-2=0 \Leftrightarrow$
		$3*y*x=2 \Leftrightarrow$
		$y*x = 2/3$

But $y*x = 2/3$ has no integer solutions for x and y . So this constraint is unsolvable and no test case exists for this test requirement.

- (d) **(10 points)** While trying to build glass-box test cases, Danny discovered one major and one minor error in his program. Carefully explain what these are by using reasoning about the code behaviour. Give constraints on the input variables x, y which specify exactly when each of these two programming errors occurs.

Model Answer: The major error is that the code has an infinite loop. This occurs whenever $x*y < 0$ as $3*x*y$ is then negative, and x is decremented by 2 inside the loop forever without $x==0$ ever becoming true. Also, when $x*y$ is odd i.e. $x*y \bmod 2 = 1$ then $3*x*y$ is always odd. In this case decrementing x by 2 always gives an odd value, so $x==0$ can never be satisfied and the loop executes infinitely often.

The minor error is that the node n5 is unreachable for any input. For the loop only terminates when $x*y \geq 0$ and $x*y$ is even. In this case, immediately after loop termination $z' = y + 3*y*x$. So, the test $z' == w' + y'$ is $y + 3*y*x = 3*y*x + y$ (note w and y are never modified in the loop). This test is tautologically true, so the next node must be n6 and n5 can never be reached. (Clearly n5 is never reached in an infinite loop either). So, the code at n5 is superfluous, though it would not lead to any crashes.

- (e) **(5 points)** Eventually Danny realized a third serious error existed in his program. It didn't actually compute the function $f(x,y) = 3xy$ for all integers x,y . What mathematical function

$g(x,y) = ?$ did Danny's code really compute? Give a detailed and precise explanation of your reasoning.

Model Answer: The function $g(x,y)$ implemented by Danny's code is not defined for all possible integer values, although it does work sometimes. From the above analysis we can see that either the algorithm does not terminate, or if $x*y \geq 0$ and $x*y$ is even then the algorithm terminates and returns w' which from above is $3*y*x$. So the function $g(x,y)$ defined by Danny's code is:

$$G(x,y) = \begin{cases} 3xy & \text{if } x*y \geq 0 \text{ and } (x \bmod 2 = 0 \text{ or } y \bmod 2 = 0) \\ \text{undefined} & \text{otherwise.} \end{cases}$$